

Análise de Código Java

Ordenação Topológica — Algoritmo de Kahn

1. Objetivo Geral

O código implementa a **Ordenação Topológica** de um grafo dirigido acíclico (DAG) utilizando o **Algoritmo de Kahn**. O objetivo é produzir uma sequência linear dos vértices tal que, para toda aresta $u \rightarrow v$, o vértice u apareça antes de v . Esta técnica é amplamente utilizada em escalonamento de tarefas, resolução de dependências (compiladores, gestores de pacotes) e análise de fluxo em pipelines.

2. Lógica e Algoritmo

O algoritmo opera em quatro etapas:

Etapa	Descrição
1 — Cálculo do in-degree	Percorre a lista de adjacência e conta, para cada vértice, quantas arestas chegam a ele (grau de entrada).
2 — Inicialização da fila	Todos os vértices com in-degree igual a zero (sem dependências) são inseridos numa fila BFS.
3 — Processamento BFS	A cada iteração, retira-se um vértice da fila, adiciona-o à ordem e decrementa o in-degree dos seus vizinhos. Vizinhos que atingem zero entram na fila.
4 — Detecção de ciclos	Se a lista resultante não contiver todos os n vértices, o grafo possui ciclo e retorna-se uma lista vazia.

3. Conceitos e Recursos Java

O código utiliza exclusivamente a **API de Coleções do Java**: **List<List<Integer>>** representa o grafo como lista de adjacência com genéricos; **Queue<Integer>** implementada por **LinkedList** garante processamento FIFO da BFS; e **ArrayList** acumula a ordenação final. O método é **estático**, sem estado de instância, favorecendo uso utilitário. O retorno **List.of()** (imutável, Java 9+) sinaliza falha por ciclo de forma idiomática.

4. Complexidade

Dimensão	Complexidade	Justificativa
Tempo	$O(V + E)$	Cada vértice e cada aresta são processados exatamente uma vez.

Espaço	$O(V + E)$	Array in-degree (V), fila (V) e lista de resultado (V), além da representação do grafo (E).
---------------	------------	---

5. Decisões de Design e Limitações

Ponto positivo: a detecção de ciclo é elegante e sem custo adicional — basta comparar o tamanho da lista final com n . **Limitação principal:** o grafo é representado por índices inteiros (0 a $n-1$); vértices com rótulos arbitrários exigiriam um mapeamento externo. O método não identifica *quais* vértices formam o ciclo. Por fim, a ordenação não é determinística quando múltiplos vértices têm in-degree zero simultaneamente — a ordem de inserção na fila dita o resultado, relevante em aplicações que requerem ordenação canônica.